哈尔滨工业大学（深圳）

面向对象的软件构造导论
实验指导书

实验二 简单工厂模式和工厂方法模式

2026 春

目录

1. 实验目的.....3
2. 实验环境.....3
3. 实验内容.....3
4. 实验步骤.....3
4.1 简单工厂模式应用.....3
4.1.1 道具创建场景分析.....3
4.1.2 代码实现.....4
4.2 工厂方法模式应用.....6
4.2.1 敌机创建场景分析.....6
4.2.2 代码实现.....7
5. 本次迭代开发目标.....9

1. 实验目的

1. 深入理解简单工厂与工厂方法模式的设计意图及结构
2. 结合案例，规范绘制两种模式的 UML 类图
3. 掌握重构方法，通过代码实现并应用两种工厂模式

2. 实验环境

1. IntelliJ IDEA 2025.3
2. OpenJDK 25

3. 实验内容

1. 深入学习简单工厂模式与工厂方法模式的设计原理，结合实例完成两种模式的 UML 结构图绘制；
2. 对系统中的敌机、道具创建逻辑进行代码重构：
   - ✓ 采用简单工厂模式实现道具对象创建
   - ✓ 采用工厂方法模式实现敌机对象创建
3. 完成本次迭代开发清单中的全部任务

4. 实验步骤

4.1 简单工厂模式应用

4.1.1 道具创建场景分析

在飞机大战游戏中包含 5 种道具：加血道具、火力道具、超级火力道具、炸弹道具、冰冻道具。所有道具均在特定敌机被击毁时按预设概率随机掉落生成，生成后以固定速度向屏幕下方移动，英雄机碰撞后自动触发对应效果，不同道具的效果存在差异（如提升火力、恢复生命、清除敌机等）。

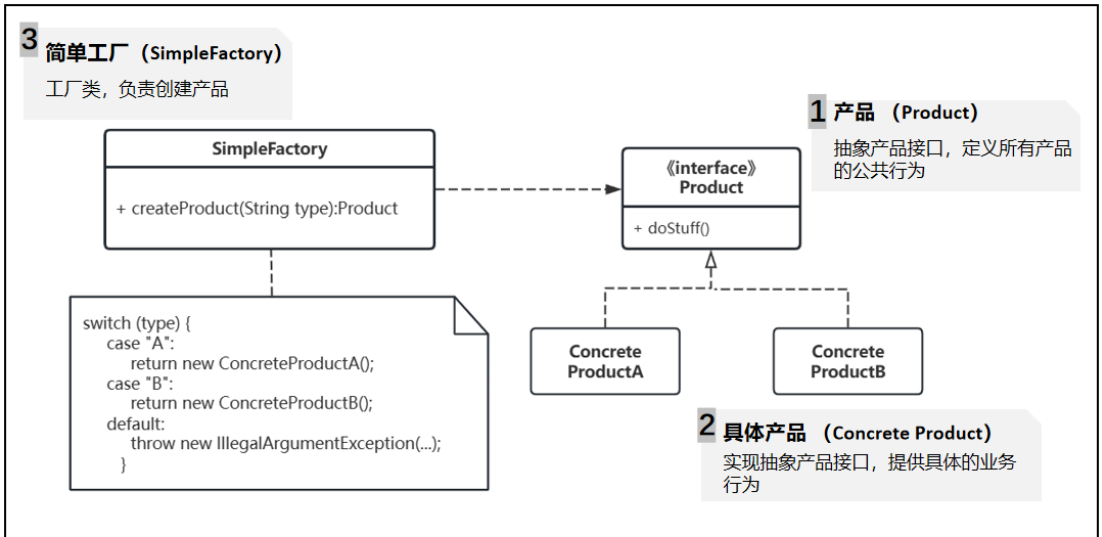
设计任务：

鉴于道具种类固定、创建逻辑集中、使用接口统一的特性，请采用简单工厂模式进行道具创建类设计与代码重构。

UML 建模：绘制该模式的 UML 类图，图中需明确标注设计模式名称，并自定义合理的类名、属性名及方法名，以完整展示简单工厂模式的实现结构。

简单工厂模式

简单工厂模式（Simple Factory Pattern）属于创建型设计模式，由一个工厂类根据传入的参数，决定创建并返回哪一种产品类实例。它将对象的创建与使用分离，让客户端无需关心对象的创建细节，只需调用工厂方法即可获取所需实例。



简单工厂模式结构图

请参考以上 UML 结构图，绘制飞机大战中道具创建场景的简单工厂模式。

4.1.2 代码实现

简单工厂模式的代码示例：

步骤 1： 创建一个接口，充当产品角色。

Product.java

```
public interface Product {
    void show();
}
```

步骤 2：创建实现接口的实体类，充当具体产品角色。

ConcreteProductA.java

```
public class ConcreteProductA implements Product {
    @Override
    public void show() {
        System.out.println("I am product A.");
```

```
    }
}
```

ConcreteProductB.java

```
public class ConcreteProductB implements Product {
    @Override
    public void show() {
        System.out.println("I am product B.");
    }
}
```

步骤 3：创建一个简单工厂类，负责创建产品。

注：简单工厂模式中，如果工厂类没有状态、不需要被继承、只是用来统一创建对象，那么工厂方法可以写成 static。

SimpleFactory.java

```
public class SimpleFactory {
    public static Product createProduct(String type) {
        switch (type) {
            case "A":
                return new ConcreteProductA();
            case "B":
                return new ConcreteProductB();
            default:
                throw new IllegalArgumentException("Unknown product type！");
        }
    }
}
```

步骤 4：使用 SimpleFactoryPatternDemo 来演示简单工厂模式的用法

SimpleFactoryPatternDemo.java

```
public class SimpleFactoryPatternDemo {
    public static void main(String[] args) {

        Product productA = SimpleFactory.createProduct("A");
        productA.show();

        Product productB = SimpleFactory.createProduct("B");
        productB.show();
    }
}
```

步骤 5：执行程序，输出结果：

```
I am product A.
I am product B.
```

4.2 工厂方法模式应用

4.2.1 敌机创建场景分析

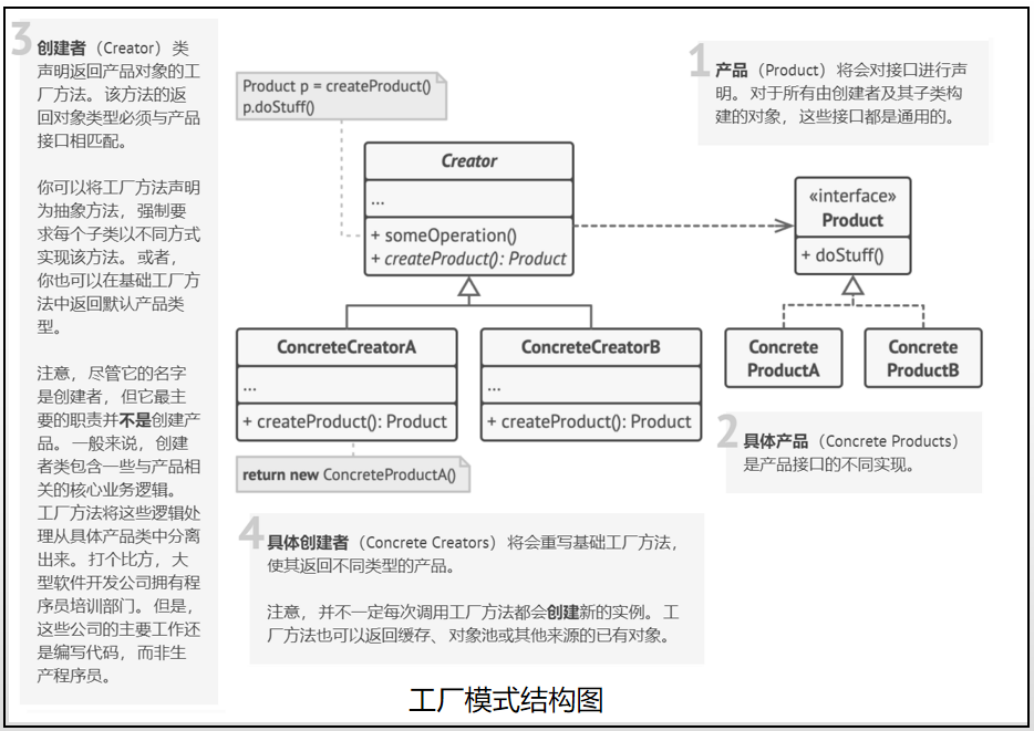
在飞机大战游戏中，敌机是核心作战单位，目前主要涵盖 5 种不同类型：普通敌机、精英敌机、精锐敌机、王牌敌机和 Boss 敌机。不同类型敌机在生命值、攻击力、移动方式、子弹弹道、得分奖励、生成概率等方面均存在明显差异，且游戏运行过程中会持续、随机生成各类敌机，生成逻辑分散且复杂。另外，随着游戏版本的迭代，未来极可能引入更多新型敌机，系统必须具备良好的可扩展性。

设计任务：鉴于敌机种类多样、结构差异明显、创建逻辑复杂、未来可能持续扩展的特性，采用工厂方法模式进行敌机创建类设计与代码重构，将对象创建与使用分离，满足高扩展需求。

UML 建模：绘制该模式的 UML 类图，图中需明确标注设计模式名称，并自定义合理的类名、属性名及方法名，完整展示工厂方法模式的实现结构。

工厂方法模式

工厂方法模式（Factory Method Pattern）也是一种创建型设计模式， 其在父类中提供一个创建对象的方法， 允许子类决定实例化对象的类型。



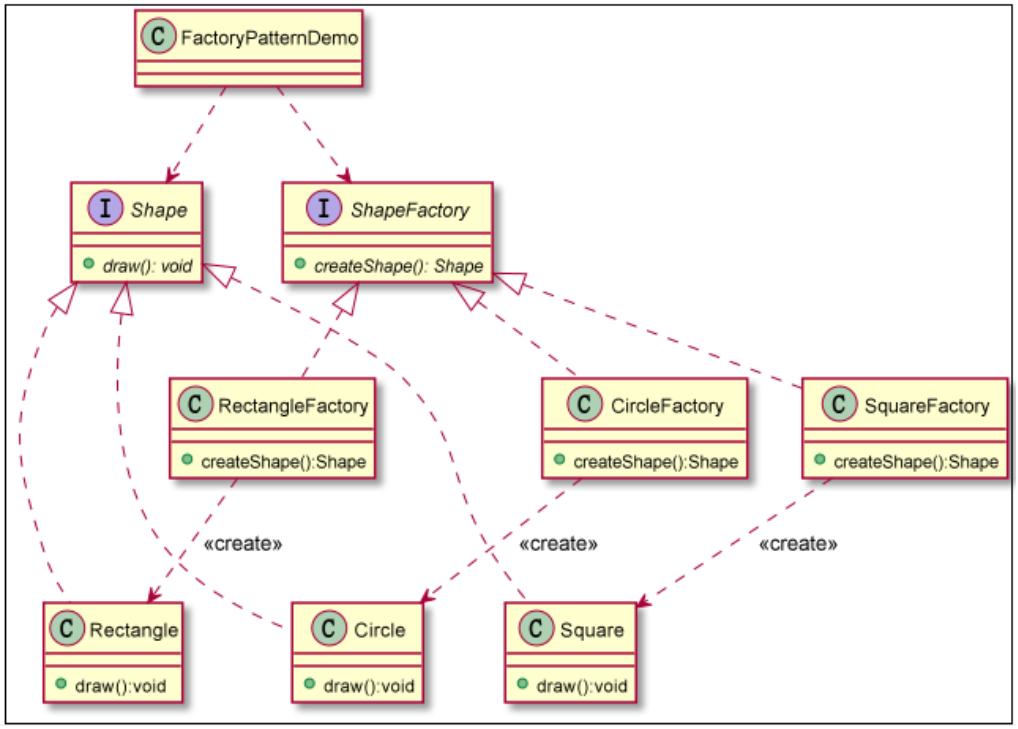
工厂模式结构图

请参考以上 UML 结构图，绘制飞机大战中敌机创建场景的工厂方法模式。

4.2.2 代码实现

工厂方法模式代码示例：

我们将创建一个 Shape 接口和实现 Shape 接口的实体类。下一步是定义工厂类 ShapeFactory。客户端 FactoryPatternDemo 类使用 ShapeFactory 来获取不同的 Shape 对象。



步骤 1： 创建一个接口，充当产品角色。

Shape.java

```
public interface Shape {
    void draw();
}
```

步骤 2：创建实现接口的实体类，充当具体产品角色。

Rectangle.java

```
public class Rectangle implements Shape {

    @Override
    public void draw() {
        System.out.println("Inside Rectangle::draw() method.");
    }
}
```

Square.java

```
public class Square implements Shape {
```

```
    @Override
    public void draw() {
        System.out.println("Inside Square::draw() method.");
    }
}
```

Circle.java

```
public class Circle implements Shape {

    @Override
    public void draw() {
        System.out.println("Inside Circle::draw() method.");
    }
}
```

步骤 3：创建一个工厂接口，充当创建者角色。

ShapeFactory.java

```
public interface ShapeFactory {

    public abstract Shape createShape();

}
```

步骤 4：创建实现工厂接口的具体工厂类，充当具体创建者角色。

RectangleFactory.java

```
public class RectangleFactory implements ShapeFactory {

    @Override
    public Shape createShape() {
        return new Rectangle();
    }
}
```

SquareFactory.java

```
public class SquareFactory implements ShapeFactory {

    @Override
    public Shape createShape() {
        return new Square();
    }
}
```

CircleFactory.java

```
public class CircleFactory implements ShapeFactory {

    @Override
    public Shape createShape() {
        return new Circle();
    }
```

```
}
```

步骤 5：使用 FactoryPatternDemo 来演示工厂方法模式的用法。

FactoryPatternDemo.java

```
public class FactoryPatternDemo {

    public static void main(String[] args) {

        ShapeFactory shapeFactory ;
        Shape shape;

      //获取 Circle 的对象，并调用它的 draw 方法
      shapeFactory = new CircleFactory();
      shape = shapeFactory.createShape();
      shape.draw();

        //获取 Rectangle 的对象，并调用它的 draw 方法
      shapeFactory = new RectangleFactory();
      shape = shapeFactory.createShape();
      shape.draw();

        //获取 Square 的对象，并调用它的 draw 方法
      shapeFactory = new SquareFactory();
      shape = shapeFactory.createShape();
      shape.draw();

    }
}
```

步骤 6：执行程序，输出结果：

```
Inside Circle::draw() method.
Inside Rectangle::draw() method.
Inside Square::draw() method.
```

5. 本次迭代开发目标

本次迭代任务清单（Checklist）：

（1）**UML 类图绘制：**按照规范绘制简单工厂模式与工厂方法模式的结构类图；

（2）**设计模式应用：**完成代码重构，使用简单工厂模式实现道具创建，使用工厂方法模式实现敌机创建；

（3）**敌机生成：**界面上可观察到 4 种敌机（Boss 敌机本次迭代不要求出场）按周期随机出现；

（4）**敌机移动：**精锐和王牌敌机向屏幕下方移动，且可左右移动；

（5）**敌机攻击：**精锐敌机按设定周期向下直射双排子弹，王牌敌机采用扇形散射弹道，单次同时发射 3 颗子弹；

（6）**道具生成：**精锐敌机、王牌敌机被击毁时按概率随机掉落某一种道具；其中精锐敌机可掉落 4 种道具（不含冰冻），王牌敌机可掉落全部 5 种道具；

（7）**道具效果：**除加血道具外，其余道具生效时在控制台输出提示信息即可。